

Render-Derived Node Context for Grounded SVG Editing

A clean text-only A/B study of counterfactual visual tags

Research question	Can a compact visual description attached to each SVG node help a language model select the node named by an editing instruction?
Control	DOM skeleton only (<code>skeleton_patch</code>).
Treatment	The identical skeleton plus cached per-node visual fields (<code>visual_stats_patch</code>).
Frozen model	Qwen 2.5 7B through Ollama, temperature 0, text input only.
Current evidence	Two completed 1,000-call A/B matrices over the same 100 source SVGs, one unconstrained and one with schema-constrained decoding, plus a mediation reanalysis, a truncation audit, an 8-instruction pilot, and a 39-source controlled holdout.

Core finding. The visual fields carried useful spatial signal in the controlled holdout, but injecting them into every patch prompt did not transfer. The official matrix first showed exact target selection falling from **56.4%** to **48.8%**. Most of that gap proved to be instrumentation: the longer prompts crossed a context limit **3.6×** as often, and schema-constrained decoding removes the format drift entirely. Re-run constrained, both arms gain 8 to 12 points and the deficit is **3.6 points** ($p = 0.057$). A real **5.5-point** grounding regression survives on the 200 node-grounding cases, concentrated in one task. No instruction in this benchmark needs what the visual fields supply, so measuring benefit requires a new held-out set.

Contents

1	Executive summary	2
2	Problem statement	2
3	Relationship to LayerSVG	3
4	Method	3
5	Clean A/B boundary	5
6	Sun and cloud case study	6
7	Experimental setup	7
8	Results	9
9	Interpretation	14
10	Limitations and failure modes	15
11	Next steps	16
12	Reproducibility	16
13	Conclusion	17

1 Executive summary

The editing model must solve two distinct problems:

1. **Grounding:** identify which SVG node corresponds to the object described by the user.
2. **Editing:** apply a valid operation to that node, such as changing fill, adding a contour, or removing the element.

An ordinary protected DOM skeleton exposes structure, tags, styles, and safe geometry summaries, but it intentionally replaces heavy path coordinates with hashes. A language model can therefore see that four nodes are red paths without knowing which one is at the top-left. The implemented treatment closes that gap by measuring each node’s visible contribution and adding a small `visual` object to its existing JSON record.

The node-hiding operation is a measurement trick, not the edit. A temporary copy of the SVG is rendered with one node hidden. The pixel difference from the full render describes that node. The temporary SVG is discarded. The model later emits a patch, and the executor applies that patch once to the original SVG.

The control and treatment share the same model, prompt function, scene serialization, parser, validator, executor, dataset order, and evaluator. The only treatment change is the attached `visual` field. This makes the experiment an interpretable A/B test of additional semantic grounding context, not a comparison between unrelated editing systems.

The small diagnostic experiments show that the measurements contain usable information. The 10-case smoke improved exact targets from 70% to 90%; an 8-instruction path-only pilot improved from 0% to 75%; and the 39-source controlled holdout improved exact targets from 7.7% to 33.3%. The holdout had 10 visual-only paired wins and no skeleton-only wins.

The completed official matrix changes the system conclusion. Across 500 matched cases per arm, exact target selection fell from 56.4% to 48.8%, exact patch match fell from 19.2% to 17.8%, valid output fell from 55.2% to 52.0%, and failure-aware MSE rose from 0.4538 to 0.4886. On the 200 node-grounding cases, exact targets fell from 79.5% to 66.0%. The treatment did yield small gains on some root-only operations, especially transparency and upside-down edits.

Three follow-up analyses then took that headline apart. A mediation analysis showed the exact-target metric was absorbing output failures: 65 of the 78 skeleton-only wins are cases where the treatment emitted no parseable patch or an unrenderable SVG, and only 13 are valid patches aimed at the wrong node. An audit of the retained call metadata then found why. Prompts above roughly 4,096 tokens were being silently truncated, the treatment’s longer prompts crossed that limit 3.6 times as often, 90 cases against 25, and every truncated call failed. On the 410 cases where both arms received a complete prompt, the same run reads -2.7 points at $p = 0.294$.

A re-run with schema-constrained decoding closes the argument. Responses with no parseable patch fall from 103 to one, both arms gain 8 to 12 points of exact-target accuracy, failure-aware raster error falls by 25% and 31%, and completion tokens equalise, confirming the earlier 68% excess was malformed continuation rather than work. The remaining overall gap is 3.6 points at $p = 0.057$.

What survives is a genuine grounding cost, smaller than the headline and larger than the mediation estimate. On the 200 grounding cases, where both arms now reach 100% valid output, the treatment loses 5.5 points across 18 paired losses that are all well-formed patches aimed at the wrong node, and the effect is concentrated in `change_color`. Render-derived context should therefore be routed through a constrained, instruction-gated ranking stage rather than appended to every prompt, and constrained decoding should be the default for both arms. The benefit side remains unmeasured here, because no instruction in this benchmark asks for the position or size the visual fields describe.

2 Problem statement

2.1 Why SVG source alone is not always enough

SVG source is structured, but the structure is not equivalent to visual meaning. A node may be:

- a path whose coordinates are intentionally hidden from the model;
- nested under inherited styles and transforms;
- partly or fully covered by a later-painted element;

- one of several nodes with the same tag and color; or
- visible only through a small, anti-aliased region.

For an instruction such as “add an outline to the small red shape in the top-left,” a protected skeleton may expose four indistinguishable red `path` nodes. DOM order is not a reliable spatial description. A whole rendered image tells the model what the composition looks like, but it does not establish a deterministic link from image pixels back to node IDs. The required context is therefore *per-node visual attribution*.

2.2 Scope of this work

This report evaluates a lightweight method for selecting existing nodes. It does not claim to solve full semantic segmentation, raster-to-SVG generation, or arbitrary reconstruction of missing geometry. The principal question is:

If every SVG node carries compact render-derived information about what it contributes, can a frozen text-only model ground natural-language editing instructions more accurately?

3 Relationship to LayerSVG

LayerSVG studies a different but adjacent problem: converting a raster image into complete, semantically separate, independently editable SVG layers [1]. Its top-down elimination pipeline selects foreground masks, inpaints regions hidden behind them, and then vectorizes the recovered layers. The paper emphasizes two properties that are directly relevant here:

- edits should operate on semantically meaningful objects rather than isolated strokes; and
- a lower layer should be complete, so moving or deleting an upper layer does not reveal a hole.

Those principles clarify the boundary of our approach. Render-difference tags can identify *which existing node* is the cloud and how it appears. They cannot recover sun geometry that was never present in the source. LayerSVG uses generative inpainting while constructing editable layers; SVGPatchLab starts from an existing SVG and performs a surgical edit. For the latter, generation should be an exceptional completion step, not the default node-selection mechanism.

LayerSVG also provides a useful efficiency precedent in its stroke-reduction analysis: it assigns a unique RGB identifier to each stroke, composites the layers, and counts which identifiers remain visible. SVGPatchLab already has a related ID-buffer implementation. For safe, opaque SVGs, one normal render and one unique-color render can recover leaf ownership, bbox, area, position, and observed color in a total of two rasterizations, independent of node count. Opacity, blending, filters, masks, and other unsupported effects route to the more general hide-and-diff method.

Table 1: LayerSVG and the present system address adjacent stages.

	LayerSVG	SVGPatchLab visual tags
Input	Raster image	Existing SVG and edit instruction
Primary goal	Build semantic, complete layers	Ground an instruction to existing node IDs
Removal role	Iterative layer decomposition plus inpainting	Temporary counterfactual measurement
Output	Newly vectorized layered SVG	Validated patch applied to original SVG
Generation	Central to hidden-region completion	Optional and guarded for genuinely missing geometry

4 Method

4.1 Counterfactual visual measurement

Let $R(S)$ be a deterministic rasterization of SVG S , and let $S \setminus n_i$ be a temporary copy in which node n_i is forced to `display:none!important`. For a pixel p , the contribution mask is

$$M_i(p) = \mathbf{1} [\|R(S)(p) - R(S \setminus n_i)(p)\|_\infty > \tau].$$

The implementation uses a 64 by 64 transparent render for compact measurement. From M_i , it calculates:

- **bbox**: the affected pixel bounds projected into SVG viewBox coordinates with `preserveAspectRatio` letterboxing accounted for;
- **area_pct**: the percentage of the canvas affected by hiding;
- **position**: a coarse spatial label derived from the contribution centroid, such as top-left, center, or bottom;
- **color**: a quantized RGB mode over contributing pixels; and
- **visible**: false when no unique pixel delta exceeds the threshold.

All measurements are cached by SVG content, render parameters, and a versioned format key. A full SVG is rendered once and each eligible node is hidden and rendered once on a cold cache. Subsequent uses read the attached statistics from disk.

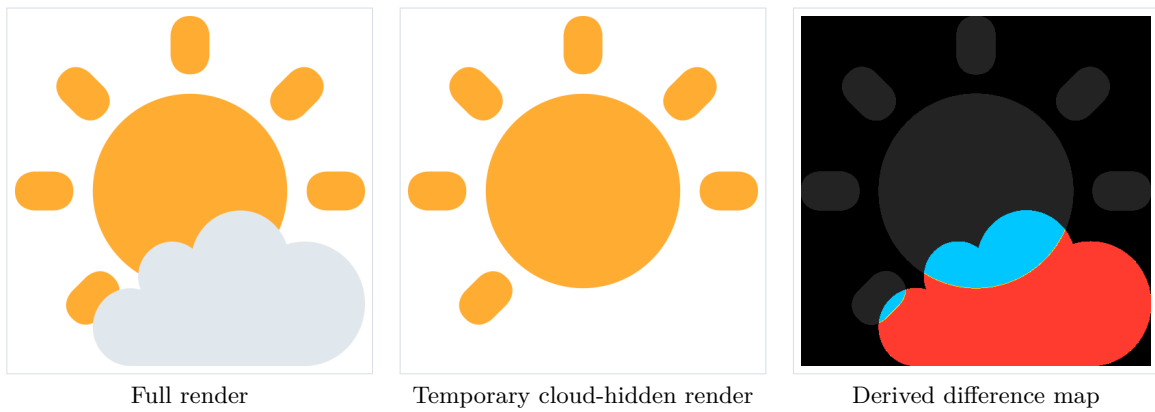


Figure 1: Counterfactual attribution on the real SVGEEditBench 1f324 sun-cloud SVG. In the difference map, **cyan** marks pixels where the orange underlayer is revealed and **red** marks pixels that become transparent; unchanged content remains dim gray. The middle and right panels are measurement artifacts, not model edits.

4.2 What the model sees

The heavy path coordinates remain protected. The new information is a compact field attached to the node that already has a stable ID:

```
{
  "id": "n7",
  "parent": "n0",
  "tag": "path",
  "attributes": {"fill": "#ff0000"},
  "protected_geometry": {
    "d": {"sha256": "...", "characters": 20}
  },
  "resolved_style": {"fill": "#ff0000"},
  "visual": {
    "bbox": [5.0, 5.0, 10.0, 10.0],
    "area_pct": 1.0,
    "position": "top-left",
    "color": "#ff0000"
  }
}
```

The model receives the instruction and the JSON skeleton as text. It matches phrases such as “small,” “red,” and “top-left” to the fields, emits a patch targeting n7 with stroke #000000, and the existing validator and executor apply that patch to the unmodified source.

4.3 Meaning of `visible:false`

`visible:false` means only that hiding the node caused no measurable unique pixel contribution at the selected resolution and threshold. It is useful as weak negative evidence, but it is not proof that the node is semantically irrelevant or fully occluded. The same observation can arise when:

- an identical-colored overlayer masks the change;
- the visual effect is below the raster threshold;
- the node is extremely small or faint; or
- a renderer does not reproduce an SVG feature faithfully.

4.4 Three-hop implementation path

Measure	<code>get_or_compute(svg)</code>	→	full and node-hidden renders produce cached statistics
Attach	<code>build_scene(svg, visual_stats=stats)</code>	→	each node receives its visual field
Use	<code>json.dumps(scene)</code> plus instruction	→	model patch, validation, and execution on the original SVG

4.5 Recommended production strategy

The current recommended measurement design is a hybrid, not a universal use of either method:

Table 2: Recommended visual-attribution strategy for an SVG with N nodes.

Path	Render cost	Use when	Role
Unique-color ID buffer	2 total	Opaque, supported, leaf-oriented SVG	Fast default attribution
Hide and compare	$N + 1$ cold, cached afterward	Complex compositing, group semantics, or unsupported features	Faithful fallback and audit
Generator completion	Model call only after deletion trigger	Required geometry is absent, not merely covered	Reconstruct missing underlayer

The current reported A/B intentionally retains the hide-and-diff treatment to keep its meaning fixed across all pilot results. The ID-buffer fast path should be introduced as a separately named treatment and tested for descriptor equivalence before it replaces the slower cache fill in production.

The completed official matrix also changes how the resulting fields should be consumed. Attribution can be measured and cached broadly, but the four visual fields should not be appended to every patch-planning prompt. They should be activated only when an instruction requires ambiguous node grounding, then consumed by a constrained target-ranking stage whose selected IDs are passed to the patch planner.

5 Clean A/B boundary

The treatment architecture inherits from the control and overrides only `scene_for()`. Both arms use:

- the same prompt template and JSON serialization;
- the same Qwen 2.5 7B model configuration and deterministic temperature;
- no image input, including when the adapter advertises image support;
- the same patch parser, policy, validator, and executor;
- the same cases in the same order; and
- the same structural and raster evaluators.

Regression tests strip **visual** from the treatment scene and assert exact equality with the control scene. They also assert identical request metadata, absence of image payloads, and identical SVG output when the model response is held fixed. Consequently, observed differences can be attributed to the additional text fields, subject to normal nondeterminism in the serving stack.

Excluded comparison. `visual_skeleton_patch` sends a whole rendered image and is not the treatment evaluated here. Including it would change both input modality and architecture, destroying the one-variable A/B boundary.

6 Sun and cloud case study

The sun-cloud example separates node grounding from geometry completion.

6.1 Case A: the sun is structurally complete

If the SVG contains a full sun circle painted below a cloud, the correct edit is simply `remove_element(cloud)`. The hidden part of the sun already exists; normal SVG painter order reveals it after cloud removal. A generator is unnecessary and could damage unrelated geometry.

6.2 Case B: the sun geometry is incomplete

If the sun was flattened, clipped, merged with the cloud, or authored only as a visible crescent, removal exposes a hole. The node selector can still identify the cloud, but the existing source does not contain enough geometry to restore the missing region. This is where a small generator can be useful.

The implemented completion path is deliberately conservative:

1. select candidate nodes using per-node semantic and visual context;
2. apply the requested deletion;
3. trigger completion only when deletion creates a sufficiently large transparent hole rather than revealing a valid underlayer;
4. ask Qwen to replace one preserved reconstruction candidate with one SVG element; and
5. accept the candidate only if it preserves the canvas, contains no unsafe SVG features, does not restore the deleted element, changes the intended region, and stays below an outside-region error threshold.

In the live smoke artifact, the final stored SVG contains a completed circle centered at (18, 18), with radius 10 and fill `#FFAC33`. The accepted attempt reported outside-region MSE 0.00178 against a maximum of 0.01, while its inside-region and generated-change checks exceeded their minimum thresholds. This is a useful mechanism check, not yet a quantitative completion benchmark.

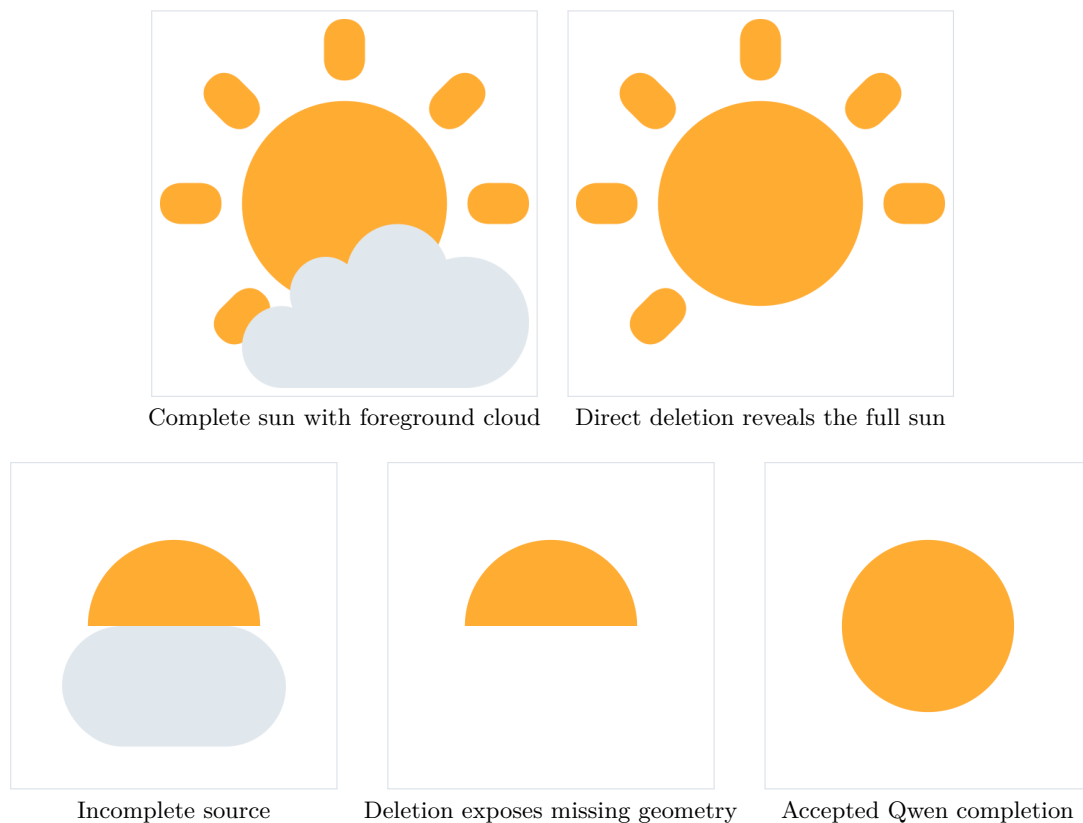


Figure 2: The deletion/completion decision rendered from retained SVG artifacts. The top row is the normal case: deletion alone is sufficient. In the bottom row, the source encodes only a half-sun; deletion exposes the missing region, so the guarded completion replaces the preserved sun candidate with a full circle.

7 Experimental setup

7.1 Frozen model and rendering

Model	Qwen 2.5 7B served by Ollama through an OpenAI-compatible adapter
Sampling	Temperature 0, maximum 768 completion tokens
Input modality	Text only for both A/B arms
Visual-stat render	64 by 64 pixels, transparent background
Evaluation render	72 by 72 pixels
Context limit	Approximately 4,096 tokens in the unconstrained run. Longer prompts were silently truncated; see Section 8.7
Constrained variant	Same server with <code>structured_output</code> set to <code>response_format</code> , sending the patch JSON Schema with every request
Server validation	114 tests pass with the Linux Cairo renderer, including the render suite skipped on the authoring machine

7.2 Metrics

The primary grounding metric is exact equality between the set of target node IDs in the predicted patch and the gold target set. Additional metrics are:

- target precision and recall;
- exact equality with the gold patch signature, including operation, target set, and attributes, rather than

byte-for-byte JSON equality;

- valid SVG output rate;
- failure-aware raster mean squared error (MSE), where lower is better and a missing, invalid, or unrenderable output receives 1.0;
- paired wins and losses on the same case; and
- prompt-token and model-latency cost.

7.3 Official smoke

The official smoke uses 10 matched SVGEEditBench task cases: two each from color change, contour, upside-down, transparency, and crop. These cases reuse two unique source SVGs across five instructions, so they are not 10 independent visual samples. The smoke checks compatibility across the normal task mix, but only four task cases are node-grounding tasks.

7.4 Completed official matrix

The full paired matrix contains 500 cases per arm: 100 cases each for color change, contour, crop-to-half, transparency, and upside-down edits. The same 100 source SVGs recur across the five task families, so the run contains 500 matched task cases but 100 unique visual sources. Color change and contour form the 200-case node-grounding group. Crop, transparency, and upside-down are 300 root-only controls whose correct target is the SVG root; they test whether extra node metadata disrupts operation planning and output validity.

Both arms ran sequentially in fixed order on the same server. Paired accuracy and error comparisons are therefore direct within each case, while latency is reported descriptively because server load and arm order were not counterbalanced.

7.5 Path-only spatial probe

The probe was constructed to remove baseline shortcuts:

- four nodes are all `path` elements with the same red fill;
- each protected `d` string has the same character count;
- coordinates are exposed only as opaque hashes;
- two unique DOM layouts permute the mapping between node order and position; and
- each instruction selects one of the four corners.

This probe directly asks whether the treatment exposes spatial information unavailable in the control context.

7.6 Expanded 39-source holdout

The expanded suite removes the pilot’s source-reuse weakness. It contains 39 matched cases and 39 distinct source SVGs:

- 27 position cases cover all nine labels in the 3 by 3 spatial grid, with three independently generated sources per label;
- 12 size cases cover smallest, second-smallest, second-largest, and largest, again with three sources per label;
- every source uses a distinct deterministic DOM permutation, balanced target-node indices, small coordinate jitter, and aspect variation;
- all candidates remain same-fill `path` nodes, raw coordinates are SHA-only, and every protected path string has the same character count; and
- a manifest freezes seed 20260725, prompt v4, the prompt-template hash, source and answer hashes, and the expected target for every case.

Runtime validation fails closed if a target crosses its spatial bin, relative size order changes, path coordinates leak, or the active prompt differs from the frozen version.

8 Results

8.1 Official 10-case matched smoke

Table 3: Official smoke results. Percentages are over 10 matched task cases from 2 unique source SVGs.

Metric	Skeleton	+ visual tags	Change
Exact target set	70%	90%	+20 points
Exact patch	20%	50%	+30 points
Valid output	60%	90%	+30 points
Mean failure-aware MSE	0.4058	0.1002	-75.3%
Prompt tokens	17,694	21,279	+20.3%
Mean model latency	9.42 s	9.99 s	+6.1%

For exact targets there were two visual-only wins, no skeleton-only wins, seven cases correct in both arms, and one correct in neither. For exact patches there were three visual-only wins and no skeleton-only wins. Failure-aware MSE favored the visual arm on four cases, the skeleton arm on none, with six ties.

This result is encouraging but not a strong grounding test: the four official color and contour cases had 100% exact target selection in both arms because their rectangle and circle coordinates were visible in the ordinary skeleton.

8.2 Completed 1,000-call official matrix

Table 4: Completed official results over 500 matched cases per arm. Exact-patch rates use all-case denominators from the paired analyzer.

Metric	Skeleton	+ visual tags	Change
Exact target set	56.4%	48.8%	-7.6 points
Exact patch	19.2%	17.8%	-1.4 points
Valid output	55.2%	52.0%	-3.2 points
Mean failure-aware MSE	0.45383	0.48862	+7.7%
Prompt tokens	1,091,351	1,212,938	+11.1%
Completion tokens	39,687	66,767	+68.2%
Mean model latency	12.12 s	17.69 s	+46.0%

For exact target selection, 78 cases were skeleton-only wins and 40 were visual-only wins; 204 were correct in both arms and 178 in neither. The exact two-sided paired sign calculation on the 118 discordant cases is $p = 0.00060$. This is strong directional evidence within the fixed benchmark, not a population estimate for unseen SVGs. Exact patch outcomes were closer: 45 skeleton-only wins and 38 visual-only wins, with no meaningful directional separation.

Table 5: Official subgroup results. Grounding contains color and contour; root-only contains crop, transparency, and upside-down.

Group	Metric	Skeleton	+ visual	Change
Grounding, $n = 200$	Exact target	79.5%	66.0%	-13.5 points
	Exact patch	5.5%	1.5%	-4.0 points
	Valid output	95.0%	81.0%	-14.0 points
	Failure-aware MSE	0.06380	0.20473	+220.9%
Root-only, $n = 300$	Exact target	41.0%	37.3%	-3.7 points
	Exact patch	28.3%	28.7%	+0.3 points
	Valid output	28.7%	32.7%	+4.0 points
	Failure-aware MSE	0.71385	0.67788	-5.0%

Table 6: Official task results; each task contains 100 paired cases.

Task	Exact target		Exact patch		Valid output	
	Skel.	Visual	Skel.	Visual	Skel.	Visual
Change color	84%	66%	11%	3%	95%	81%
Set contour	75%	66%	0%	0%	95%	81%
Crop to half	33%	33%	32%	22%	33%	33%
Transparency	25%	16%	4%	12%	4%	12%
Upside down	65%	63%	49%	52%	49%	53%

The task split explains why the overall patch rate moved only slightly while grounding regressed. The visual arm produced more valid and more exact patches for transparency and upside-down, but it substantially reduced target and output validity on color and contour. The extra JSON also increased malformed or policy-invalid completions, including JSON-Patch-like fields and disallowed operations. The treatment is therefore not a safe universal prompt addition for this 7B planner. Sections 8.7 and 8.8 show that most of this particular table’s gap is context truncation and format drift rather than node selection, and restate the comparison with both removed.

8.3 Path-only spatial grounding probe

Table 7: Spatial probe results. Percentages are over 8 matched instructions from 2 unique layout SVGs.

Metric	Skeleton	+ visual tags	Change
Exact target set	0%	75%	+75 points
Exact patch	0%	75%	+75 points
Valid output	100%	100%	no change
Mean failure-aware MSE	0.01008	0.00175	-82.6%
Prompt tokens	14,272	17,224	+20.7%
Mean model latency	10.27 s	9.60 s	-6.5%

Six instructions were visual-only target wins, none were skeleton-only wins, and both arms missed two. A naive instruction-level sign test would give $p = 0.03125$, but that calculation is not an inferential result because the instructions are clustered within only two layout SVGs. The result is therefore reported as mechanism evidence; a larger held-out set of independently generated sources is required for uncertainty estimates.

8.4 Expanded spatial and size holdout

Table 8: Expanded holdout results over 39 matched, distinct source SVGs.

Metric	Skeleton	+ visual tags	Change
Exact target set	7.7%	33.3%	+25.6 points
Exact patch	7.7%	33.3%	+25.6 points
Mean target precision	14.7%	40.3%	+25.5 points
Mean target recall	48.7%	69.2%	+20.5 points
Valid output	100%	100%	no change
Mean failure-aware MSE	0.01400	0.01031	-26.4%
Prompt tokens	92,404	116,833	+26.4%
Mean model latency	11.75 s	14.44 s	+22.9%

Table 9: Exact targets by held-out descriptor family.

Family	Skeleton	+ visual tags	Visual-only wins
Position, 27 sources	2/27 (7.4%)	11/27 (40.7%)	9
Relative size, 12 sources	1/12 (8.3%)	2/12 (16.7%)	1
Overall, 39 sources	3/39 (7.7%)	13/39 (33.3%)	10

Across exact targets, 10 sources were visual-only wins, none were skeleton-only wins, three were correct in both arms, and 26 were correct in neither. An exact two-sided McNemar/sign test on the 10 discordant pairs gives $p = 0.001953$. Position alone had nine visual-only and no skeleton-only wins ($p = 0.003906$); size had only one and zero ($p = 1.0$). These values are descriptive for one fixed synthetic generator, seed, prompt, and model run. They establish directional consistency within this controlled suite, not population inference or an effect size for natural SVGs. Raster error was lower for the visual arm on 14 sources, lower for the skeleton arm on four, and tied on 21.

The residual errors are informative. All outputs were valid, so the remaining problem is grounding behavior rather than patch execution. All 78 patches used the correct operation, stroke color, and stroke width. Of 26 visual-arm target errors, 10 selected every candidate, five selected a partial multi-node set, and 11 selected one wrong path. The wrong singleton was always the residual default **n2** or **n5**. The generic contour rule still competes with spatial and relative-size modifiers. Visual metadata helps substantially, especially for position, but a four-field tag does not by itself make a 7B patch planner a reliable ranking model.

The latency increase is descriptive only because the arms ran sequentially and were not counterbalanced. Prompt tokens are the clean cost measurement.

8.5 Prompt sensitivity

A shared experimental prompt explicitly instructed the model to combine visual descriptors conjunctively. The retained spatial-v2 artifact shows that it raised the blind spatial baseline from 0% to 37.5%, while the treatment remained at 75%. The completed official matrix used the frozen prompt v4 and still showed negative transfer on the broad task mix. Together, these results show that both arms are prompt-sensitive and that a ranking-specific prompt must be evaluated as a separately named architecture rather than folded into the current treatment after the fact.

8.6 Mediation analysis: output validity versus grounding

Table 5 reports a 13.5-point exact-target deficit and a 14.0-point valid-output deficit on the same 200 grounding cases. Those metrics are not independent: a malformed or policy-rejected response yields no usable target set and is therefore scored as a target miss. A post-hoc reanalysis of the retained JSONL streams separates the two effects.

Table 10: Exact-target selection reported marginally, then conditioned on valid output. Conditioning removes almost all of the deficit.

Group	<i>n</i>	Exact target		Exact target valid	
		Skel.	Visual	Skel.	Visual
Overall	500	56.4%	48.8%	88.8%	88.5%
Grounding	200	79.5%	66.0%	83.7%	81.5%
Root-only	300	41.0%	37.3%	100.0%	100.0%
Change color	100	84.0%	66.0%	88.4%	81.5%
Set contour	100	75.0%	66.0%	78.9%	81.5%

The overall 7.6-point deficit becomes 0.3 points. The grounding deficit falls from 13.5 points to 2.2. Root-only cases are exactly 100% in both arms once output is valid, because their correct target is trivially the SVG root; their entire exact-target signal is output validity and carries no grounding information whatsoever.

Table 11: Paired target outcomes restricted to cases where both arms produced valid output.

Group	n	Skel.	Visual	Skel.-only	Visual-only	p
Overall	216	89.8%	86.1%	13	5	0.096
Grounding	162	86.4%	81.5%	13	5	0.096
Root-only	54	100.0%	100.0%	0	0	1.000
Change color	81	91.4%	81.5%	9	1	0.021
Set contour	81	81.5%	81.5%	4	4	1.000

On the 216 cases where both arms produced valid output there are 13 skeleton-only and five visual-only target wins, an exact two-sided sign calculation of $p = 0.096$. The $p = 0.00060$ reported in Section 8.2 does not survive the restriction.

Decomposing the 78 skeleton-only target wins by what the visual arm actually produced:

Table 12: Cause of each skeleton-only exact-target win.

Group	Wins	No patch	Invalid SVG	Wrong target
Overall	78	32	33	13
Grounding	32	19	0	13
Root-only	46	13	33	0

Sixty-five of the 78 losses, or 83%, are output failures rather than target-selection failures.

The dominant failure class is drift toward RFC 6902 JSON Patch. The error `PatchError: unknown patch fields: ['op', 'path', 'value']` rose from 25 cases in the control to 92 in the treatment, and responses with no parseable patch at all rose from 39 to 103. A representative failure emits standard JSON Patch instead of the project schema:

```
[{"op": "replace", "path": "/0/attributes/fill", "value": "#FFCC4D"}]
```

Total errored responses barely moved, from 224 to 240. What changed is composition: schema drift rose by 67 cases while policy violations fell by 51. Given a longer and denser JSON scene, the model falls back to the patch dialect it saw in pretraining rather than the dialect the prompt specifies. The treatment trades policy errors for format errors, and format errors are scored as grounding misses.

Conditioning caveat. `valid_output` is a post-treatment variable, so this is a mediation analysis rather than an unbiased estimate of the treatment effect on grounding. The both-valid subset is selected: it contains the cases where the treatment arm behaved, which may be systematically easier. The analysis identifies which causal path carries the observed damage. It does not prove that the visual fields are harmless.

Two results survive conditioning. First, `change_color` shows a genuine grounding regression: 9.9 points on the both-valid subset, nine skeleton-only against one visual-only win, $p = 0.021$ over $n = 81$. Second, `set_contour` shows none at all, at 81.5% in both arms with a four-four split. The aggregate claim of regression across both node-grounding tasks therefore holds for one task, not two.

Confirmed and corrected by Section 8.8. The per-task prediction held exactly under constrained decoding: `change_color` regressed by 10 points and `set_contour` by 1. The magnitude did not. This section’s residual estimate of 2.2 points on the grounding group was optimistic; the clean measurement is 5.5. The both-valid subset was easier than the full set, exactly as the conditioning caveat warned.

8.7 A context-window confound in the official matrix

Re-reading the retained `model_call_details` while preparing the constrained re-run exposed a defect in the official matrix that neither the original analysis nor Section 8.6 accounted for.

Reported prompt-token counts are not well behaved at the top of the size distribution. Cases whose prompts fit report plausible varying counts up to 4,035 tokens. Above that, counts collapse to 2,050 or 0, values far too small

for the text sent. The same cases in the re-run report 4,241 to 4,874 tokens. The boundary and the constant are consistent with a 4,096-token context limit truncating the serialized scene mid-JSON.

Truncation is identified by comparing the reported count against the prompt actually sent rather than by matching those constants, because one case legitimately tokenizes to 2,050. Intact prompts in this run span 2.78 to 3.66 characters per reported token and truncated ones 5.76 to 6.18, so a threshold of 4.5 separates them with roughly 20% margin either side. By that criterion 25 skeleton calls and 90 visual calls were truncated.

Every one of those calls failed. All 25 and all 90 produced no patch, no valid output, and no correct target. They account for 25 of the skeleton arm’s 39 no-patch failures and 90 of the visual arm’s 103. The RFC 6902 drift catalogued in Section 8.6 is therefore concentrated almost entirely in truncated prompts: a model shown a scene cut off mid-structure falls back to the patch dialect it knows.

This matters for the A/B because the treatment’s prompts are longer by construction, so more of them crossed the limit: 90 cases against 25, a factor of 3.6. Restricting the official matrix to the 410 cases where both arms received a complete prompt changes the headline result.

Table 13: The official matrix with truncated calls excluded.

Subset	<i>n</i>	Exact target	Paired	<i>p</i>
All cases	500	56.4% → 48.8%	78 / 40	0.00060
Complete prompts only	410	62.2% → 59.5%	51 / 40	0.294

The −7.6-point deficit becomes −2.7, and $p = 0.00060$ becomes $p = 0.294$. Valid output reverses sign, from −3.2 points to +4.4. The grounding group falls from −13.5 to −6.1 points. The strongest statistical claim in the original matrix was substantially an artifact of one arm overflowing a context window more often than the other.

8.8 Constrained decoding re-run

The patch format was given a JSON Schema with `additionalProperties` false at both levels, an operation enum restricted to the four implemented operations, and `targets` required for the three that address existing nodes. Servers supporting structured output receive it and cannot decode anything else. Two separately named architectures, `strict_skeleton_patch` and `strict_visual_stats_patch`, inherit their scene construction unchanged from the unconstrained pair, so the completed matrix stays reproducible and the treatment contrast is untouched. The prompt text is byte-identical to the frozen v4 prompt.

Table 14: Constrained re-run over 500 matched cases per arm, Qwen 2.5 7B through Ollama 0.32.3 with schema-constrained decoding.

Metric	Skeleton	+ visual tags	Change
Exact target set	64.0%	60.4%	-3.6 points
Exact patch	28.2%	25.6%	-2.6 points
Valid output	66.6%	67.2%	+0.6 points
Mean failure-aware MSE	0.33953	0.33762	-0.6%
Prompt tokens	1,170,922	1,491,227	+27.4%
Completion tokens	33,792	33,697	-0.3%
Mean model latency	1.16 s	1.27 s	+9.4%

Both arms improved substantially against their unconstrained counterparts. Exact target selection rose by 7.6 points for the control and 11.6 for the treatment. Failure-aware MSE fell by 25.2% and 30.9%. Responses with no parseable patch fell from 39 to **0** in the control and from 103 to **1** in the treatment. Completion tokens, which the unconstrained run inflated by 68.2% in the treatment arm, are now within 0.3% of each other, confirming that the excess was malformed continuation rather than reasoning. Mean latency fell from 12.12 to 1.16 seconds because the model no longer emits prose preamble.

Table 15: Constrained subgroup and task results.

Group	Exact target		Valid output	
	Skel.	Visual	Skel.	Visual
Grounding, $n = 200$	82.0%	76.5%	100.0%	100.0%
Root-only, $n = 300$	52.0%	49.7%	44.3%	45.3%
Change color	87%	77%	100%	100%
Set contour	77%	76%	100%	100%
Crop to half	32%	32%	32%	31%
Transparency	26%	19%	3%	7%
Upside down	98%	98%	98%	98%

Overall the paired split is 49 skeleton-only against 31 visual-only wins, $p = 0.057$, no longer separating at the conventional threshold. On the grounding group it is 18 against 7, $p = 0.043$, which does. Because validity on the grounding group is now 100% in both arms, the marginal and conditional rates coincide and the 5.5-point deficit is measured directly rather than inferred.

The decomposition of the 49 skeleton-only wins is the cleanest result in this report, because the two failure kinds have separated completely by task family:

- **0** emitted no patch, against 32 unconstrained;
- **31** produced a patch that yielded an invalid SVG, and every one is a root-only case; and
- **18** produced a valid patch aimed at the wrong node set, and every one is a grounding case.

Two independent routes now agree on the size of the real effect. Excluding truncated calls from the old run gives -2.7 points overall and -6.1 on grounding; the constrained re-run gives -3.6 and -5.5 . Constrained decoding also lifted **upside_down** from 65% and 63% to 98% in both arms, because the schema makes the unimplemented **transform** operation unrepresentable and the model falls back to a legal one.

9 Interpretation

9.1 What the evidence supports

The render-derived measurements contain real spatial information. When the ordinary skeleton deliberately removes coordinate shortcuts and the instruction names position or relative size, the controlled 39-source holdout improved exact targets by 25.6 points, with 10 visual-only wins and no skeleton-only wins. This establishes mechanism validity: the hide-and-compare payload can connect visual descriptors to stable node IDs.

The completed official matrix establishes a second, equally important result: the usefulness of the measurement does not imply that unconditional prompt injection improves a general patch planner. In the fixed model and dataset, target accuracy, exact patches, validity, and failure-aware raster error all moved against the treatment overall.

Two follow-up analyses then narrowed what that regression means, and the second corrected the first.

The mediation analysis showed the measured cost was dominated by output format compliance rather than node selection. The truncation audit (Section 8.7) then explained why: the treatment’s longer prompts crossed a 4,096-token context limit 3.6 times as often as the control’s, and every truncated call failed. On complete prompts alone the official matrix reads -2.7 points at $p = 0.294$, not -7.6 at $p = 0.00060$.

The constrained re-run (Section 8.8) confirms this from the other direction. With decoding pinned to the patch schema, unparseable responses fall to zero, both arms gain 8 to 12 points of exact-target accuracy, and the remaining gap is -3.6 points overall at $p = 0.057$.

What survives is smaller than the headline and larger than the mediation estimate. On the 200 grounding cases, where both arms now reach 100% valid output, the treatment loses 5.5 points, 18 paired wins against 7, $p = 0.043$. Every one of those losses is a valid patch aimed at the wrong node, and the effect is concentrated in **change_color**. The defensible statement is therefore that most of the original regression was an artifact of context truncation and format drift, and that beneath it lies a genuine but smaller grounding cost on one task family.

9.2 What the evidence does not support

The evidence does not support deploying `visual_stats_patch` as the universal replacement for `skeleton_patch`. The four fields compete with operation constraints and patch-format instructions inside a limited 7B model’s context. More context is not automatically better context.

The current fields do not provide object names such as “cloud,” “sun,” or “tree.” They make words like “small,” “red,” and “top-left” groundable. Semantic object recognition still comes from tag/style clues, composition, future learned labels, or a separate vision-language stage. Likewise, counterfactual rendering measures visible contribution, not the complete amodal shape of an occluded object. The 33.3% held-out exact rate also shows that metadata availability is not equivalent to reliable model use.

9.3 Efficiency

In the unconstrained run, the treatment used 1,212,938 prompt tokens versus 1,091,351, an 11.1% increase, and completion tokens increased 68.2%. Both figures are now known to be distorted: truncated calls under-report their prompts, and the completion excess was malformed continuation rather than work. The constrained re-run gives the clean measurement. Prompt tokens are 1,491,227 against 1,170,922, a **27.4%** increase, which is the real context cost of the four visual fields once nothing is silently cut off. Completion tokens are within 0.3% of each other. Mean latency is 1.27 against 1.16 seconds; the arms still ran sequentially, so latency remains descriptive.

Rendering cost is front-loaded and cacheable. For repeated instructions on the same SVG, the warm-cache cost is a JSON lookup rather than a fresh render per node. Cold-cache and warm-cache preprocessing time should be reported separately when the ID-buffer and guarded-ranking architectures are compared.

10 Limitations and failure modes

1. **Repeated official sources.** The full matrix has 500 task cases per arm but 100 unique source SVGs reused across five task families. Paired case counts must not be interpreted as 500 independent visual samples.
2. **Fixed model, prompt, and arm order.** The result is one temperature-zero run of Qwen 2.5 7B with skeleton first and visual second. Model generality and causal latency require replication and counterbalancing.
3. **Synthetic holdout.** Its ambiguity is controlled, but all candidates are generated red paths under one contour task and one prompt wording. It does not capture the object, compositing, and authoring diversity of real SVGs.
4. **The unconstrained matrix is not a clean instrument.** Its headline result combined three effects: context truncation that hit the longer-prompted arm 3.6 times as often, format drift toward RFC 6902, and a genuine grounding difference. Only the third is about the treatment’s hypothesis. Constrained decoding removes the second and eliminates the first for these inputs, but the original 500-case numbers should not be quoted without that qualification.
5. **Silent truncation was not detected at run time (now fixed).** Nothing in the harness flagged a prompt that exceeded the server’s context window; the failure surfaced only as an unusual token count during a later audit. The adapter now compares reported prompt tokens against the text actually sent and raises `PromptTruncatedError` when the ratio implies the server saw less than was given. Architectures deliberately let it propagate rather than recording it as a model mistake, so a misconfigured context aborts the run instead of being scored as a wrong answer. Replayed against all 1,000 retained calls the check flags 115 of the 115 truncated ones with no false positives. It remains a heuristic: a tokenizer with markedly larger tokens would need `max_chars_per_prompt_token` raised.
6. **Conditioning on a post-treatment variable understated the effect.** The mediation analysis conditions on `valid_output`, which the treatment affects, so the both-valid subset is selected rather than randomized. Its 2.2-point residual estimate for the grounding group proved optimistic against the 5.5 points measured directly under constrained decoding, though its per-task pattern held exactly.
7. **Unique contribution, not full shape.** The difference mask describes pixels that change when a node is hidden. Covered regions are not recovered.
8. **Raster sensitivity.** Very small, faint, filtered, or anti-aliased features can fall below the measurement threshold at 64 pixels.
9. **Renderer fidelity.** Filters, masks, blend modes, markers, external resources, and fonts can vary across renderers.

10. **Groups and ownership.** Hiding a group can measure the union of descendants, while hiding one leaf measures only that leaf. Prompts must not confuse containment with semantic identity.
11. **No learned object labels.** Nodes with similar color, area, and position remain ambiguous when the instruction names an object class.
12. **Generation risk.** A completion model can hallucinate, duplicate, or shift geometry. It must remain trigger-gated and validator-gated.
13. **Incomplete system provenance.** The holdout manifest records the seed, prompt hash, source hashes, and expected targets, but hardware, git commit, cache state, and server load still need automatic capture.

11 Next steps

The constrained re-run has completed, so the prerequisite is discharged and the list is reordered around what it left standing:

1. **Measure the hypothesis on instructions that can test it.** All 200 grounding instructions in this benchmark identify their target by fill color, which the ordinary skeleton already carries, and the remaining 300 address the root. No case requires the position or size the visual fields supply, so the treatment’s ceiling here is a tie and its floor is the 27.4% context cost now measured cleanly. A held-out set of natural position- and size-naming instructions over real authored SVGs is the only way to observe a benefit, and is now the blocking experiment;
2. **Diagnose change_color.** It carries essentially the whole surviving regression, 87% to 77%, while `set_contour` under an identical scene loses one point. Both arms reach 100% valid output there, so this is a pure selection effect and should be readable from the 18 paired losses;
3. gate visual context by instruction type, omitting it for root-only operations, whose exact-target scores carry no grounding information once output is valid;
4. separate target ranking from patch planning: a constrained first stage returns only node IDs, then the existing planner receives those IDs and the instruction without the full visual payload;
5. fix the remaining root-only failures, which are now the largest single block: 31 of the 49 paired losses are patches that parse and validate but render an invalid SVG, concentrated in crop and transparency;
6. ablate compact descriptors, including size rank and numeric centroid, instead of assuming the current four-field block is optimal;
7. benchmark the two-render ID-buffer fast path against hide-and-diff for descriptor equivalence, preprocessing speed, and unsupported-feature fallback;
8. repeat the paired matrix with counterbalanced arm order, multiple seeds, natural grounding instructions, partial overlap, groups, and real authored path shapes; and
9. construct a deletion/completion benchmark with known complete and incomplete underlayers, recording cold-cache and warm-cache timing.

12 Reproducibility

12.1 Official paired run

```
python -m svgpachlab.cli matrix \
  --config configs/experiments/skeleton_patch.json \
  --model-config configs/models/qwen2.5-7b-ollama.json \
  --architectures skeleton_patch visual_stats_patch \
  --render \
  --output-root runs/qwen2.5-7b-visual-stats-ab-full-v1-server
```

The detached server run completed with 500 records in each arm. The output directory named in the command above contains both JSONL result streams, per-arm summaries, `matrix-summary.json`, and `paired-analysis.json`. The execution log is retained at `runs/visual-stats-next-steps-v1.log`.

12.2 Paired analysis

```
python scripts/analyze_visual_stats_ab.py \
  --root runs/qwen2.5-7b-visual-stats-ab-full-v1-server \
  --dataset-root SVGEditBench
```

12.3 Constrained decoding re-run

```
python -m svgpatchlab.cli matrix \
  --config configs/experiments/skeleton_patch.json \
  --model-config configs/models/qwen2.5-7b-ollama-constrained.json \
  --architectures strict_skeleton_patch strict_visual_stats_patch \
  --render \
  --output-root runs/qwen2.5-7b-constrained-ab-full-v1-server
```

The model config differs from the unconstrained one by a single field, **structured_output**, set to **response_format**. The adapter then sends **PATCH_JSON_SCHEMA** with every request. Servers advertising the older vLLM interface take **guided_json** instead; off restores the original behaviour. Both analyzers accept **-arms** so the strict directories can be read:

```
python scripts/analyze_visual_stats_ab.py \
  --root runs/qwen2.5-7b-constrained-ab-full-v1-server \
  --arms strict_skeleton_patch strict_visual_stats_patch
```

The run completed 1,000 calls in roughly 18 minutes on an Ollama 0.32.3 host, against several hours unconstrained, because the model no longer emits prose around its answer.

12.4 Validity-conditioned mediation analysis

```
python scripts/analyze_conditional_validity.py \
  --root runs/qwen2.5-7b-visual-stats-ab-full-v1-server \
  --dataset-root SVGEditBench
```

This reads the same two JSONL streams and writes **conditional-validity-analysis.json** beside them. It reuses the gold-target derivation of the paired analyzer, so exact-target definitions are identical across both reports; the only additions are the validity conditioning, the both-valid paired restriction, the loss decomposition, and the error-class tally.

12.5 Pilot spatial probe

```
python scripts/run_spatial_grounding_probe.py \
  --model-config configs/models/qwen2.5-7b-ollama.json \
  --output-root runs/qwen2.5-7b-spatial-grounding-probe-v1
```

12.6 Expanded held-out suite

```
python scripts/run_spatial_grounding_holdout.py \
  --model-config configs/models/qwen2.5-7b-ollama.json \
  --output-root runs/qwen2.5-7b-spatial-grounding-holdout-v1 \
  --seed 20260725
```

13 Conclusion

Render-derived attribution is a useful measurement, but unconditional visual metadata injection is not the production solution. The controlled holdout proved that bbox, area, position, and color can recover spatial information hidden by the protected skeleton. The completed official matrix then showed that feeding those fields to every Qwen patch request reduced exact target selection from 56.4% to 48.8% and increased model cost. Two audits have since established that most of that gap was instrumentation rather than grounding.

The longer treatment prompt crossed a 4,096-token context limit 3.6 times as often as the control's, and every truncated call failed outright. Excluding those cases, the same run reads -2.7 points at $p = 0.294$. Re-running under schema-constrained decoding, unparseable responses fall from 103 to one, both arms gain 8 to 12 points of accuracy, failure-aware raster error drops by 25% and 31%, and the remaining gap is -3.6 points at $p = 0.057$.

A real effect does survive, smaller than the headline and larger than the mediation estimate. On the 200 grounding cases, with both arms now at 100% valid output, the treatment loses 5.5 points, and every one of those 18 paired losses is a well-formed patch aimed at the wrong node. The cost is concentrated in `change_color`; `set_contour` loses one point.

The recommended route remains a guarded pipeline: measure and cache attribution, invoke it only when the instruction requires ambiguous node selection, resolve candidate IDs with a constrained ranking stage, and pass the selected IDs to the ordinary patch planner, with root-only operations bypassing the visual payload. Constrained decoding should now be the default for every arm, since it improved the control as much as the treatment.

The honest limit of this evidence is that the benchmark cannot reward the idea. Every grounding instruction here names its target by fill color, which the plain skeleton already carries, so the visual fields can only ever cost 27.4% more prompt context and never repay it. The 5.5-point loss is the price of irrelevant context, not evidence that render-derived attribution fails at what the holdout showed it can do. Establishing benefit requires instructions that name position or size on real authored SVGs, which is the next experiment rather than a robustness check.

For the sun-cloud example, the decision is now precise. Use render-derived context to select and remove the cloud. If the full sun exists underneath, stop. Only when deletion reveals genuinely missing geometry should the guarded Qwen completion path reconstruct the preserved sun node.

References

- [1] Xuancheng Liu, Teng Hu, Ran Yi, Chenyang Shi, Wenbo Li, Yu-Kun Lai, and Paul L. Rosin. *LayerSVG: Layer-wise Semantic Editable Scalable Vector Graphics Synthesis*. ICLR 2026 withdrawn submission, submitted 15 September 2025. <https://openreview.net/forum?id=4lEZLpXKkq>. The supplied PDF is the anonymous double-blind manuscript.